Especificação Base de Projetos

ENSO ORIGINS LDA | Outubro 2026

Este documento define as regras base comuns a todos os projetos de software da ENSO. Serve de referência para a especificação de cada novo projeto, sendo depois complementado com os requisitos próprios de cada caso.

1. Tipo de software

- Aplicações web, na totalidade ou quase.

2. Back-end

- Tipicamente desenvolvido em Node.

3. Base de dados

A escolha da base de dados é flexível e decidida caso a caso, conforme o software a especificar o justifique.

- Postgres como opção típica.
- Bases de dados não relacionais quando o caso o justifique.
- Timescale para séries temporais.
- SQLite para projetos pequenos, quando é suficiente.
- Evitam-se sempre soluções Microsoft, como o SQL Server.

4. Front-end

- Tipicamente baseado em React.
- Será entregue mais tarde um repositório de UI de base, para seguir os componentes já usados noutros projetos.

5. Tecnologia de execução

Utiliza-se Docker Compose em ambiente de desenvolvimento e, eventualmente, também em produção.

- Um serviço por componente: front-end, back-end, base de dados e o que mais for necessário.
- Os dados persistentes são sempre colocados numa pasta data, seguida do nome do serviço (data/nome-do-servico).
- Nunca se usam named volumes do Docker, exceto quando haja uma razão muito específica que o justifique.

6. Utilizadores

Existe sempre uma componente de utilizadores, com perfis diferenciados.

- Tipicamente um perfil de administrador e um perfil normal.
- Por vezes um perfil de super administrador.

7. Zonas da aplicação

Na maior parte dos casos, a aplicação divide-se em duas zonas:

- Um front-end público, sem autenticação.
- Um back-office, em rota do tipo /admin, com autenticação.

8. Autenticação

- Login sempre com e-mail e password. Não se utiliza username.
- OTP sempre disponível.
- Idealmente, também autenticação por impressão digital, suportada no próprio computador.
- Login e reset de password sempre presentes, com os formulários típicos de recuperação.
- O utilizador tem sempre forma de alterar a sua própria password no seu perfil.

9. Settings

Existe sempre uma zona de settings no back-office.

Envio de e-mail, incluindo sempre:

- Host.
- Protocolo, tipicamente SMTP.
- Utilização ou não de SSL.
- Ignorar ou não certificados.
- Respetiva autenticação.

Branding, onde se configuram:

- Logótipo do projeto ou do cliente, apresentado na página de login e utilizado como favicon.
- Nome do cliente, quando aplicável.
- Título do projeto.

Estas variáveis de branding ficam disponíveis para utilização no front-end e no back-end.

10. E-mails do sistema

- Todos os e-mails enviados pela plataforma (boas-vindas, reset de password, avisos e outros) são em formato HTML.
- Os e-mails mantêm sempre o branding do projeto: logótipo, nome do cliente e título do projeto, conforme configurado nas settings.

11. Gestão de utilizadores no back-office

- Opção de enviar um e-mail de boas-vindas a um utilizador, a indicar que pertence à plataforma.
- Em cada envio do e-mail de boas-vindas, escolhe-se se inclui ou não a opção de reset de password.

12. Interface

- Tipo de letra: Inter, em toda a aplicação.

- O software é sempre responsivo em todas as zonas: front-end público, back-office e páginas de login, reset de password e OTP.
- O favicon é o logótipo configurado na secção de branding.

Formatação de datas:

- Datas sempre no formato YYYY/MM/DD (exemplo: 2026/10/05).
- Quando é necessária a hora, acrescenta-se hh:mm (exemplo: 2026/10/05 14:30).
- Os segundos só se apresentam quando necessários (exemplo: 2026/10/05 14:30:15).

13. Marca ENSO

- Em todos os projetos, no back-office e na página de login, aparece sempre a indicação Powered by ENSO.
- O texto "Powered by" fica numa linha e o logótipo da ENSO na linha de baixo.
- O logótipo da ENSO é obtido a partir de enso-origins.com.

14. CLAUDE.md

Todos os projetos incluem na raiz do repositório um ficheiro CLAUDE.md, que contém sempre as secções seguintes, para além da informação tecnológica própria de cada projeto:

```
## Git Workflow
- After every code change, commit it to the current branch
- Use descriptive commit messages that explain what was changed and why
- Never switch branches or create new branches unless explicitly asked
- Run any relevant checks before committing if applicable

## Behavior
- Before starting any non-trivial task, ask clarifying questions to better
  understand the requirements, edge cases, and expected outcome
- If a request is ambiguous, always ask rather than assume
- Confirm your understanding of the task before writing any code

## Communication
- Be concise in explanations unless asked for detail
- When something is unclear, ask don't assume
- Flag potential issues or risks even if not asked
```